

WORK SAMPLE | CASE STUDY

Lumo

Make hard product decisions faster, and learn from every one.

A working AI decision tool for product managers, designed, built and shipped end to end: product thinking, UX, front end, AI integration, bilingual content and production deployment.

Role	Product lead, designer and engineer (solo build)
Author	Terrance Range
Live product	https://www.trylumo.co
Status	Working prototype in production, English and Spanish
Stack	Next.js 16, React 19, TypeScript, Tailwind CSS 4, Anthropic Claude via tool calling, Upstash Redis, Vercel

Executive summary

Lumo is a web app for product managers who have to make a hard call and then explain it to everyone it touches. A PM writes down a first instinct in plain language. Lumo reads the situation back, proposes options, compares them side by side, helps the PM commit to a choice with a written trade-off and a confidence score, and then drafts a tailored message for every audience affected: the VP, the execution team, adjacent teams and customers.

The decision is saved with the reasoning behind it. Later, Lumo uses that record to show the PM where their instincts have been right and where they have been off, and it surfaces that pattern at the moment the PM is about to repeat it.

I built Lumo as a work sample to show how I move from a real problem to a shipped, polished, AI-native product. It is a prototype, but a complete one: real model calls, real persistence, real deployment, a custom domain, and a content and SEO layer behind it.

What this document covers

- The problem I saw and why it matters
- Who Lumo is for and the principles that shaped it
- What I built: the six-step flow and the long-term learning loop
- How it works technically, and the decisions behind the architecture
- Problems I hit in production and how I solved them
- What I would build next

The problem

AI gave you more options. It didn't give you more judgment.

I am a product leader and a three-time founder. I kept watching product manager friends go through the same pattern: they would make a hard decision, then spend days explaining it to every team that needed to hear it. Generative AI made the drafting faster, but it did not make the decisions better, and it never remembered how last quarter's decisions turned out.

Three specific failures sit underneath that pattern.

- **Decisions are made on instinct and never checked.** A PM states a confident view, ships, and months later nobody compares the outcome with the original belief. Without that loop, confidence

and accuracy drift apart.

- **The communication tax is huge.** One decision needs a different message for the VP, for engineering, for sales and for a customer. Writing each version by hand is slow, and the tone often goes wrong.
- **Tools start from zero every time.** A chat assistant helps with one decision, then forgets it. It does not capture the PM's first instinct before the answer is known, and it does not remember how past calls turned out.

Why existing tools fall short

Approach	What it does well	Where it breaks down
Chat assistant	Fast thinking partner for one decision	No memory of past calls, no record of the original instinct, no outcome tracking
Decision journal	Keeps the history	Entirely manual, so it rarely survives a busy week
Docs and Slack threads	Good for sharing	Reasoning is buried, nothing connects decisions to outcomes

Lumo's position: do the research on the decision in front of you, and remember how it turned out.

03

Who it is for and design principles

The target user is a working product manager who regularly makes calls with real trade-offs: sequencing work, committing dates to customers, deciding what to cut. They are time-poor, they answer to leadership, and they carry the burden of explaining their reasoning to several audiences.

Principles I set before building

Principle	How it shows up in the product
Capture the instinct first	Step 1 asks for the PM's raw take before any analysis, so it can be compared with the outcome later.
Honest AI, never invented	Screens use real model output and real user input. Demo content is clearly labeled as an example.
Zero friction to start	No account, no install, no IT approval. Decisions live in the browser.
Reduce the communication tax	One decision produces a draft for each audience, plus likely pushback with suggested replies.

Interrupt the next mistake	Patterns are shown when the PM is about to repeat one, not buried in a report.
Calm, editorial tone	Warm cream canvas, serif signature lines, no em dashes, no exclamation points, restrained motion.

04

What I built

The decision flow

The core of the app is a six-step path with a persistent numbered progress bar. Each step is backed by the model where it adds value and left editable where the PM's judgment must stay in charge.

Step	Name	What happens
1	What's happening	The PM describes the situation in their own words. Detected entities are highlighted behind an editable text area, with real counts, and the text is sent to the analysis endpoint.
2	What I'm reading	Lumo reads the situation back: the real question, what matters, who is affected, how pressing it is. Fields appear one at a time and remain editable. A closing "I noticed" line surfaces something the PM did not say.
3	Your options	AI-generated options, each marked as from the user or from Lumo, with at least one Lumo proposal always present. The PM can add their own.
4	Side by side	A comparison of each option: cost level and summary, affected parties, reversibility and risk. It stacks into cards on mobile.
5	Your choice	The PM picks a direction, writes the rationale and what they are giving up, and rates confidence from 1 to 5. A hold-to-commit gesture makes the commitment deliberate, with an instant accessible alternative.
6	Tell people	Lumo drafts a message for every affected audience in the right channel (email, Slack or DM), with up to two likely objections and suggested responses. Each draft can be rewritten.

The learning loop

Decisions are saved with sequential numbers (No.1, No.2 and so on), the original instinct, the chosen option, the trade-off and the confidence score. The "Over time" experience shows what this enables: a weeks-later moment where the PM is about to commit to a customer date and Lumo says, in effect, you have been here before.

Ten times you have been this sure about a date. Seven of them slipped. Your rule: add engineering's worst case before you give a date.

In the demo this is illustrated with a clearly labeled sample persona and a 40-decision record, plus a judgment map and a monthly brief that summarise outcomes (better than expected, as expected, worse). The goal is to show the end state of the product honestly, without pretending the sample data belongs to the visitor.

Supporting surfaces

- **Landing page** with a labeled example product panel that cycles through raw input, read-back and audience message, a dilemma ticker, a problem section, a six-step preview and a comparison with chat tools and decision journals.
- **Interactive live demo** that replays a full sample decision so visitors see the value without typing anything.
- **Product decision guides**, a content library in English and Spanish, built for search and credibility.
- **Bilingual experience** across the product and its AI output, so the model responds in the visitor's language.

05

How it is built

Layer	Choice	Why
Framework	Next.js 16 App Router, React 19, TypeScript	Server rendering for SEO and speed, route handlers for the API, strict types for AI payloads
Styling	Tailwind CSS 4 with a custom token set	Fast iteration with a consistent palette: cream canvas, persimmon accent, three text tones
Type	Inter for UI, JetBrains Mono for numbers and dates, Source Serif 4 Italic for the signature line	Clear hierarchy, with the serif reserved for four moments so it keeps its meaning
AI	Anthropic Claude via the official SDK, with forced tool calls	Structured, schema-validated output instead of free text
Persistence	Browser localStorage for past decisions	No sign-up barrier; decisions stay on the user's device

Protection	Upstash Redis rate limiting	Caps abuse and controls model cost on a public, unauthenticated tool
i18n	Locale-prefixed routes with typed English and Spanish dictionaries	The Spanish dictionary must mirror English exactly or the build fails
Hosting	Vercel with a custom domain, analytics, sitemap and robots	Zero-config deploys and fast global delivery

Three API routes

- **/api/analyze** takes the raw situation and returns the read-back, options and comparisons through the `provide_decision_analysis` tool.
- **/api/decide** takes the committed decision and returns a clarity summary, the execution team and a draft per audience through `provide_audience_drafts`, including pushback and replies.
- **/api/rewrite** rewrites a single audience draft through `rewrite_audience_draft` when the PM wants a different angle.

Reliability decisions in the AI layer

- **Forced tool calls.** The model must answer through a JSON schema, so the UI always receives predictable fields.
- **Server-side validation.** Type guards check every draft: unique audience ids, valid channels, non-empty bodies, a VP draft present, and the named execution team matching one of the audiences. Invalid output is rejected rather than shown.
- **Output cleanup and retry.** A shared cleanup function normalises model text to a human writing standard, and a retry handles transient failures.
- **One shared model constant.** The model is configured in a single file, so upgrading is a one-line change.
- **Honest failure states.** The header shows AI status (connected, not connected, checking) from the latest real call. A diagnostic panel, gated behind a flag, exposes attempt details when something fails.
- **Locale-aware generation.** The requested language is passed into the prompt so Spanish users receive Spanish drafts.

06

Problems I hit and how I solved them

A real build includes things going wrong. These are the three that taught me the most.

Problem	Root cause	Fix
The decision flow reset itself mid-step in the browser	A React key tied to view state caused the step component to remount on each update, wiping input and state	Removed the unstable key, stabilised the AI call with a memoised callback, and re-ran an end-to-end browser test through all steps
AI worked in preview but not in production	The production runtime needed an explicit, valid model configuration; the preview path had masked the gap	Centralised the model constant, wired the project's Anthropic key, added response logging and the diagnostic panel, and verified with live calls
Sitemap returned 404 on the live domain	A host loop: the platform redirected the apex domain to www while my middleware redirected www back to the apex	Made <code>www.trylumo.co</code> the single canonical host in the SEO config and middleware, redeployed and verified the sitemap on the public URL

What I took from this: test the deployed product, not just the preview; make failures visible to the user and to myself; and treat domain, redirects and canonical URLs as part of the product.

07

Craft details worth noting

- **Signature line, used four times only.** Source Serif italic with a persimmon left border appears on past decision closures, the "I noticed" line, the Home empty state and the completion screen. Restraint keeps it special.
- **Copy rules.** No em dashes, no exclamation points, no "let's" or "we'll". The voice is calm and direct, written for someone under pressure.
- **Motion with discipline.** Animations run 200 to 900 ms, ease-out, no bounce or parallax, and all respect `prefers-reduced-motion`.
- **Accessibility.** Semantic structure, labeled progress, and an instant alternative to the hold-to-commit gesture.
- **Mobile first.** Everything works at 375 px, including the stacked comparison cards.
- **No fake data presented as real.** Sample personas and the 40-decision record are labeled as examples.
- **SEO and discoverability.** Per-locale metadata, sitemap, robots, canonical host, Open Graph image and a guides library in two languages.

08

Outcome and what is next

Lumo is live, bilingual and usable end to end: a visitor can watch the demo, run their own decision through the full six steps, get audience-specific drafts, and keep a persistent record of what they decided and why.

What it demonstrates about how I work

- I start from a lived problem and write the position before the code.
- I design the product and the system together: the interface, the schema the model must obey, and the failure modes.
- I ship, test in production, and fix what the real environment reveals.
- I care about voice, restraint and honesty in AI products, not just features.

Next steps I would take

Priority	Next step	Why it matters
1	Accounts and cross-device sync	Moves the record off a single browser so the learning loop survives device changes
2	Outcome check-ins	Prompt the PM at a set date to log how the decision landed, feeding the pattern engine with real data
3	Pattern detection on real records	Replace the sample record with live analysis of the user's own history
4	Integrations	Send drafts straight to Slack and email, and pull context from tools PMs already use
5	Team mode	Shared decision logs so a group can see how its calls hold up

Lumo is my attempt at the part AI skipped: better judgment, and a memory of how it turned out.

Terrance Range | www.trylumo.co